

TD 8 : Les TRIGGERS

Table des matières

1) Problématique	1
2) Exercices	1
Exercice 1 : Mise à jour automatique des soldes lors d'un virement	1
Exercice 2 : Création automatique du login et du compte client	2
Exercice 3 : Contrôle de chevauchement des périodes de rémunération	3
3) Conclusion	4

1) Problématique

Dans une base de données relationnelle comme celle de notre application bancaire HHB-Direct, certaines règles de gestion doivent être appliquées automatiquement par le serveur de base de données, sans dépendre du code de l'application cliente.

L'objectif de ce TD est d'apprendre à créer des triggers (déclencheurs) sous PostgreSQL en PL/pgSQL. Un trigger permet d'exécuter automatiquement une fonction lors d'un événement précis (INSERT, UPDATE ou DELETE) sur une table pour garantir la cohérence des données.

2) Exercices

Exercice 1 : Mise à jour automatique des soldes lors d'un virement

1) Écriture du trigger dans Notepad++

On crée un trigger AFTER INSERT sur la table operation. Dès qu'une ligne de virement est insérée, la fonction maj_soldes_virement débite le compte émetteur et crédite le compte destinataire du montant indiqué :

```

c:\xampp\htdocs\exo1.sql - Notepad++
Fichier  Édition  Recherche  Affichage  Encodage  Langage  Paramètres  Outils  Macro  Exécution
Modules d'extension  Documents  ?
exo1.sql
1 CREATE OR REPLACE FUNCTION maj_soldes_virement()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     IF NEW.type_operation = 'virement' THEN
5         UPDATE compte
6         SET solde = solde - NEW.montant
7         WHERE num_compte = NEW.num_compte AND id_type = NEW.id_type;
8
9         UPDATE compte
10        SET solde = solde + NEW.montant
11        WHERE num_compte = NEW.num_compte_vers AND id_type = NEW.id_type_vers;
12    END IF;
13    RETURN NEW;
14 END;
15 $$ LANGUAGE plpgsql;
16
17 DROP TRIGGER IF EXISTS trig_insert_operation_virement ON operation;
18 DROP TRIGGER IF EXISTS trig_operation_virement ON operation;
19 CREATE TRIGGER trig_operation_virement
20 AFTER INSERT ON operation
21 FOR EACH ROW

```

2) Test du virement dans le terminal

On effectue un virement de 200 € du compte 1 vers le compte 3 avec la fonction virement(), puis on vérifie que les soldes ont bien été actualisés :

Résultat :

```

Console_PostgreSQL_Exo1
-- 1. Soldes avant virement :
id_type | num_compte | solde
-----+-----+-----
1 | 1 | 1000
1 | 3 | 3400
(2 lignes)

-- 2. Virement de 200 euros du compte 1 vers le compte 3 :
virement
-----
1
(1 ligne)

-- 3. Soldes apres virement (mis a jour par le trigger) :
id_type | num_compte | solde
-----+-----+-----
1 | 1 | 800
1 | 3 | 3600
(2 lignes)

```

(Le compte 1 est bien passé de 1000 € à 800 € et le compte 3 de 3400 € à 3600 €, ce qui confirme la mise à jour automatique par le trigger.)

Exercice 2 : Création automatique du login et du compte client

1) Écriture du trigger dans Notepad++

On crée un trigger BEFORE INSERT sur la table client. Lors de l'ajout d'un client, le trigger génère l'identifiant avec creer_id_internet(), lui attribue un mot de passe et crée le compte de connexion PostgreSQL avec CREATE ROLE :

```

1 CREATE OR REPLACE FUNCTION creer_client_trigger()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     NEW.identifiant_internet := creer_id_internet(NEW.nom_client, NEW.prenom_client);
5     NEW.mdp_internet := NEW.identifiant_internet;
6     IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = lower(NEW.identifiant_internet)) THEN
7         EXECUTE 'CREATE ROLE ' || quote_ident(NEW.identifiant_internet) || ' WITH LOGIN PASSWORD '' ' ||
8             NEW.mdp_internet || ''';
9     END IF;
10    RETURN NEW;
11 END;
12 $$ LANGUAGE plpgsql;
13
14 DROP TRIGGER IF EXISTS trig_insert_nouveau_client ON client;
15 DROP TRIGGER IF EXISTS trig_insert_client ON client;
16 CREATE TRIGGER trig_insert_client
17 BEFORE INSERT ON client

```

2) Test de l'insertion dans le terminal

Dans le terminal, on insère le client 'Martin Luc' sans préciser d'identifiant ni de mot de passe, puis on vérifie la table client et le catalogue pg_roles :

Résultat :

```

-- 1. Insertion du client Martin Luc sans identifiant ni mot de passe :
INSERT 0 1

-- 2. Données generees dans la table client :
 num_client | nom_client | prenom_client | identifiant_internet | mdp_internet
-----+-----+-----+-----+-----
          9 | Martin    | Luc           | Lmartin              | Lmartin
(1 ligne)

-- 3. Compte PostgreSQL cree dans pg_roles :
 rolname | rolcanlogin
-----+-----
 Lmartin | t
(1 ligne)

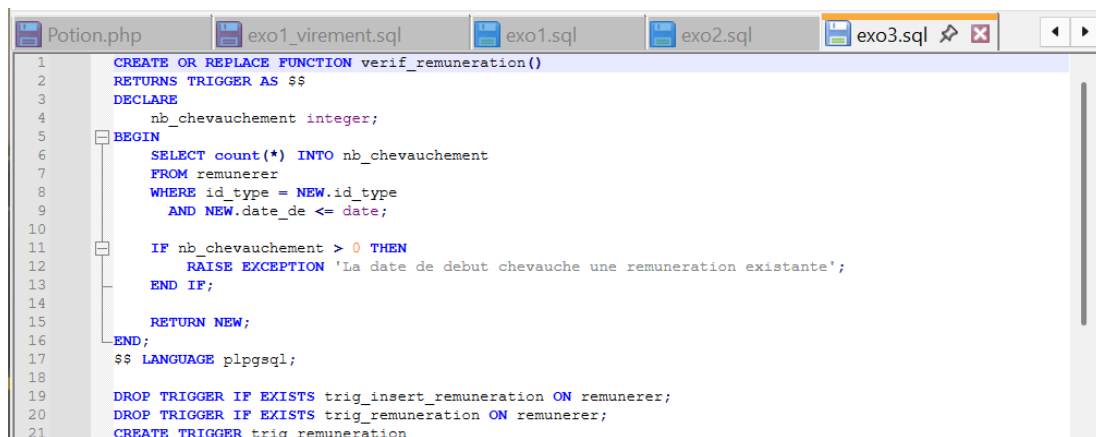
```

(L'identifiant 'Lmartin' et le mot de passe ont été générés automatiquement, et le compte PostgreSQL existe bien dans pg_roles.)

Exercice 3 : Contrôle de chevauchement des périodes de rémunération

1) Écriture du trigger dans Notepad++

On crée un trigger BEFORE INSERT sur la table remunerer. Si la date de début de la nouvelle rémunération chevauche une période déjà existante pour ce type de compte, le trigger déclenche une erreur avec RAISE EXCEPTION :

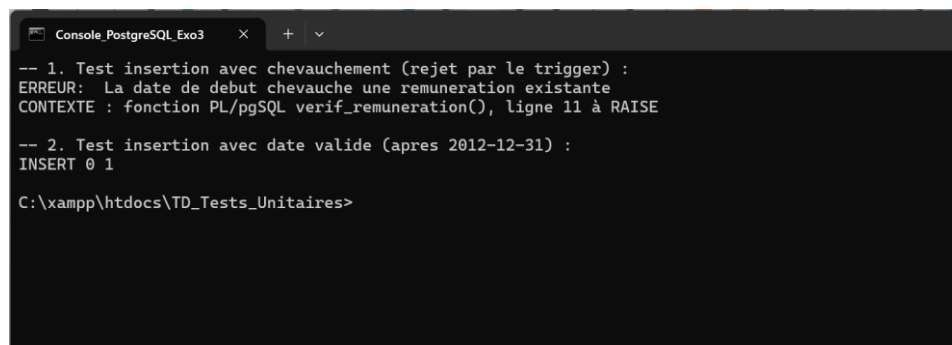


```
1 CREATE OR REPLACE FUNCTION verif_remuneration()
2 RETURNS TRIGGER AS $$
3 DECLARE
4     nb_chevauchement integer;
5 BEGIN
6     SELECT count(*) INTO nb_chevauchement
7     FROM remuneration
8     WHERE id_type = NEW.id_type
9           AND NEW.date_de <= date;
10
11     IF nb_chevauchement > 0 THEN
12         RAISE EXCEPTION 'La date de debut chevauche une remuneration existante';
13     END IF;
14
15     RETURN NEW;
16 END;
17 $$ LANGUAGE plpgsql;
18
19 DROP TRIGGER IF EXISTS trig_insert_remuneration ON remuneration;
20 DROP TRIGGER IF EXISTS trig_remuneration ON remuneration;
21 CREATE TRIGGER triq_remuneration
```

2) Test de l'insertion dans le terminal

Dans le terminal, on teste d'abord une insertion avec une date en conflit (qui doit être bloquée), puis une insertion avec une date valide :

Résultat :



```
Console_PostgreSQL_Exo3 x + v
-- 1. Test insertion avec chevauchement (rejet par le trigger) :
ERREUR: La date de debut chevauche une remuneration existante
CONTEXTE : fonction PL/pgSQL verif_remuneration(), ligne 11 à RAISE
-- 2. Test insertion avec date valide (apres 2012-12-31) :
INSERT 0 1
C:\xampp\htdocs\TD_Tests_Unitaires>
```

(Le trigger a bien bloqué l'insertion avec un message d'erreur explicite lors du chevauchement, et a validé l'insertion avec une date correcte.)

3) Conclusion

Ce TD nous a permis de comprendre le fonctionnement des triggers en PostgreSQL. Grâce aux déclencheurs AFTER et BEFORE, nous avons automatisé le calcul des soldes bancaires, la création des comptes de connexion utilisateurs et le contrôle de validité des dates directement au niveau de la base de données.